

79. Regular Expressions: Patterns aur Flags

Regular expressions text me search aur replace ka powerful tarika hain. JS me ye `RegExp` object se milte hain aur strings ke methods me integrated hain.

Regular Expression kya hai?

Ek **pattern** aur optional **flags**.

Banane ke 2 syntaxes:

```
js

regexp = new RegExp("pattern", "flags");    // "lamba" tarika

regexp = /pattern/;                          // "chhota" tarika, slashes se, bina flags
regexp = /pattern/gmi;                       // flags g, m, i ke saath
```

Slashes `/.../` JS ko batate hain ki regexp ban raha hai (jaise strings ke liye quotes). Dono me `regexp` built-in `RegExp` class ka instance hota hai.

Farak: slashes wale pattern me expressions insert nahi kar sakte (jaise template literal ka `${...}`), wo poori tarah **static** hai. Slashes tab jab likhte waqt regexp pata ho (sabse aam). `new RegExp` tab jab dynamically bani string se regexp banana ho:

```
js

let tag = prompt("What tag do you want to find?", "h2");
let regexp = new RegExp(`<${tag}>`);    // "h2" par /<h2>/ ke barabar
```

Flags

Search ko badalne wale, JS me sirf 6:

- `i`: case-insensitive (`A` aur `a` me farak nahi)
- `g`: **saare** matches dhundho, bina iske sirf pehla
- `m`: multiline mode (`^` `$` anchors wale chapter me)
- `s`: "dotall" mode, dot `.` newline `\n` ko bhi match kare (character classes chapter me)
- `u`: poora Unicode support, surrogate pairs sahi handle (Unicode chapter me)
- `y`: "sticky" mode, text me exact position par search

Search: `str.match`

3 working modes:

1. `g` flag ho: saare matches ka array

js

```
let str = "We will, we will rock you";  
alert(str.match(/we/gi));    // We,we
```

((i) ki wajah se (We) aur (we) dono mile.)

2. (g) na ho: sirf pehla match, array ki tarah, (index 0) par poora match, aur kuch extra properties:

js

```
let result = str.match(/we/i);  
  
alert(result[0]);    // We (pehla match)  
alert(result.length);    // 1  
  
alert(result.index);    // 0 (match ki position)  
alert(result.input);    // poori source string
```

Agar regexp ka koi hissa parentheses me ho to (0) ke alawa aur indexes bhi ho sakte hain (Capturing groups chapter).

3. Koi match nahi: (null) (chahe (g) ho ya na ho). Khaali array nahi, (null). Ye bhool gaye to error aata hai:

js

```
let matches = "JavaScript".match(/HTML/);    // null  
if (!matches.length) { ... }                // Error: Cannot read property 'len
```

Hamesha array chahiye to:

js

```
let matches = "JavaScript".match(/HTML/) || [];
```

Replace: (str.replace)

(str.replace(regexp, replacement)): (g) ho to saare matches, warna sirf pehla.

js

```
"We will, we will".replace(/we/i, "I");    // I will, we will
"We will, we will".replace(/we/ig, "I");   // I will, I will
```

`replacement` me special combinations se match ke hisse daal sakte hain:

Symbols	Kaam
---------	------

<code>\$&</code>	poora match
----------------------	-------------

<code>\$`</code>	match se pehle ka hissa
------------------	--------------------------------

<code>\$'</code>	match ke baad ka hissa
------------------	-------------------------------

<code>\$n</code>	<code>(n)</code> (1-2 digit) wale parentheses ka content (Capturing groups chapter)
------------------	---

<code>\$<name></code>	diye gaye naam wale parentheses ka content
-----------------------------	--

<code>\$\$</code>	<code>\$</code> character
-------------------	---------------------------

```
js
```

```
"I love HTML".replace(/HTML/, "$& and JavaScript");    // I love HTML and JavaSc
```

Test: `regexp.test`

Kam se kam ek match mile to `true`, warna `false`:

```
js
```

```
let str = "I love JavaScript";
let regexp = /LOVE/i;
alert(regexp.test(str));    // true
```

Methods ki poori jaankari "Methods of RegExp and String" chapter me.

Summary

- Regexp = pattern + optional flags: `(g)`, `(i)`, `(m)`, `(u)`, `(s)`, `(y)`.
- Flags aur special symbols ke bina regexp search normal substring search jaisa hi hai.
- `str.match(regexp)`: matches dhundhta hai `(g)` ho to sab, warna pehla).
- `str.replace(regexp, replacement)`: matches ko replace karta hai `(g)` ho to sab, warna pehla).
- `regexp.test(str)`: kam se kam ek match ho to `true`, warna `false`.